



FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

(FIAP)

**CURSO SUPERIOR DE TECNOLOGIA EM INTELIGÊNCIA
ARTIFICIAL**

GRUPO SÃO PAULO E INTERIOR

JONAS TADEU V. FERNANDES

LEVI PASSOS SILVEIRA MARQUES

RAPHAEL DA SILVA

RAPHAEL DINELLI NETO

YAN PIMENTAL COTTA

CardiolA:

IoT e Visualização de Dados para a Saúde Digital

SÃO PAULO

2026



FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

(FIAP)

**CURSO SUPERIOR DE TECNOLOGIA EM INTELIGÊNCIA
ARTIFICIAL**

GRUPO SÃO PAULO E INTERIOR

JONAS TADEU V. FERNANDES

LEVI PASSOS SILVEIRA MARQUES

RAPHAEL DA SILVA

RAPHAEL DINELLI NETO

YAN PIMENTAL COTTA

CardiolA:

IoT e Visualização de Dados para a Saúde Digital

Trabalho acadêmico apresentado ao Curso Superior de Tecnologia em Inteligência Artificial da Faculdade de Informática e Administração Paulista (FIAP), como requisito para obtenção de nota na disciplina Physical Computing, Embedded AI, Robotics & Cognitive IoT.

Tutor: Caique Nonato da Silva Bezerra

SÃO PAULO

2026



SUMÁRIO

1. INTRODUÇÃO	1
2. DESENVOLVIMENTO.....	2
2.1 Objetivo da primeira etapa.....	2
2.2 Componentes utilizados	2
2.3 Estrutura lógica de funcionamento	4
2.4 Regras de alerta implementadas.....	5
2.5 Armazenamento local com micros.....	5
2.6 Buffer em RAM como fallback e demonstração	6
2.7 Estratégia de resiliência offline	6
2.8 Sincronização dos dados.....	7
2.9 Organização do código e boas práticas adotadas	7
2.10 Justificativa para o não uso do SPIFFS e adoção de meio alternativo.....	8
3. CONCLUSÃO.....	10
FONTES	11



IoT - Internet of Things (Internet das Coisas)

RAM - Random Access Memory (Memória de Acesso Aleatório)

MQTT - Message Queuing Telemetry Transport

LED - Light Emitting Diode (Diodo Emissor de Luz)

CSV - Comma-Separated Values (Valores Separados por Vírgula)

APIs - Application Programming Interface (Interface de Programação de Aplicações)

FIFO - First In, First Out (Primeiro a Entrar, Primeiro a Sair)

SPIFFS - SPI Flash File System (Sistema de Arquivos para Memória Flash SPI)

SPI NOR - Serial Peripheral Interface NOR Flash



RESUMO

O presente trabalho apresenta o desenvolvimento da primeira etapa do projeto CardiolA, cujo objetivo é simular um sistema vestível de monitoramento cardíaco com base em conceitos de Internet das Coisas e *Edge Computing*. O protótipo foi implementado com ESP32 no ambiente Wokwi e estruturado para representar o comportamento de um smartwatch de saúde capaz de capturar sinais vitais e eventos relevantes do usuário. Foram utilizados o sensor DHT22 para leitura de temperatura e umidade, o MPU6050 para detecção de movimento, um potenciômetro para simulação de batimentos cardíacos e um botão de emergência para representar eventos críticos manuais. A solução implementa armazenamento local em cartão microSD como mecanismo principal de persistência temporária dos dados e um buffer em memória *RAM* como *fallback* em caso de indisponibilidade do armazenamento físico. Quando a conectividade simulada é restabelecida, os dados acumulados são sincronizados por meio do Monitor Serial. Dessa forma, o sistema demonstra a importância da computação em borda em aplicações críticas de saúde.

Palavras-chave: *Edge Computing*; ESP32; *IoT*; Monitoramento Cardíaco; Saúde Digital.



ABSTRACT

This paper presents the development of the first stage of the Cardiola project, whose objective is to simulate a wearable cardiac monitoring system based on Internet of Things and Edge Computing concepts. The prototype was implemented using ESP32 in the Wokwi environment and structured to represent the behavior of a health smartwatch capable of capturing vital signals and relevant user events. The DHT22 sensor was used to read temperature and humidity, the MPU6050 to detect movement, a potentiometer to simulate heart rate, and an emergency button to represent manual critical events. The solution implements local storage on a microSD card as the main mechanism for temporary data persistence and a RAM buffer as a fallback strategy in case the physical storage becomes unavailable. When simulated connectivity is restored, the accumulated data are synchronized through the Serial Monitor. Thus, the system demonstrates the importance of edge computing in critical digital health applications.

Keywords: Edge Computing; ESP32; IoT; Cardiac Monitoring; Digital Health.



1. INTRODUÇÃO

A crescente presença da Internet das Coisas em aplicações de saúde tem ampliado as possibilidades de monitoramento contínuo, coleta remota de sinais vitais e acompanhamento preventivo de pacientes em diferentes contextos. Nesse cenário, dispositivos vestíveis, como relógios inteligentes e pulseiras eletrônicas, têm assumido papel relevante ao permitir o acompanhamento frequente de variáveis fisiológicas e comportamentais, como temperatura, frequência cardíaca, atividade motora e eventos de emergência.

Entretanto, a simples transmissão contínua de dados para a nuvem nem sempre é suficiente para garantir a confiabilidade de sistemas críticos. Em ambientes com instabilidade de rede, falhas de conectividade podem comprometer a continuidade do monitoramento e causar perda de informação importantes. Por esse motivo, a computação em borda, ou *Edge Computing*, torna-se uma abordagem fundamental para o próprio dispositivo embarcado.

A primeira etapa do projeto CardiolA foi concebida justamente com esse foco. O objetivo constitui em desenvolver um protótipo funcional capaz de simular um sistema smartwatch de saúde utilizando ESP32 no ambiente Wokwi, realizando leituras periódicas de sensores e mantendo operação mesmo na ausência de conectividade. Para atender a essa proposta, a arquitetura do sistema foi planejada com armazenamento local em micros e mecanismo complementar em memória *RAM*, garantindo maior robustez na retenção temporária dos dados até sua posterior sincronização.

Assim, esta fase do projeto busca demonstrar, de forma prática, como um dispositivo de borda pode operar de maneira autônoma, coletar sinais relevantes, avaliar condições de alerta localmente e preservar dados enquanto a comunicação com a nuvem não está disponível. Além disso, esta etapa estabelece a base técnica para as próximas fases do CardiolA, que envolverão transmissão via *MQTT*, dashboards interativos e Automação de alertas.



2. DESENVOLVIMENTO

2.1 Objetivo da primeira etapa

A primeira etapa do CardiolA teve como finalidade implementar um sistema embarcado de monitoramento capaz de simular o comportamento inicial de um dispositivo vestível aplicado à saúde digital. Mais do que realizar a simples leitura de sensores, a proposta constitui em estruturar um fluxo local de coleta, processamento, armazenamento e sincronização, representando de forma prática os princípios da computação em borda.

Nesse contexto, o sistema foi projetado para funcionar em dois estados principais: modo offline e modo online. No modo offline, os dados coletados devem ser preservados localmente, sem interrupção da leitura dos sensores. No modo online, as informações pendentes devem ser transmitidas ao ambiente externo, ainda que, nesta etapa, essa transmissão seja representada de forma simulada pelo Monitor Serial. Dessa forma, o foco central do projeto não apenas no sensoriamento, mas na continuidade operacional do sistema, mesmo diante de falhas temporárias de conectividade.

2.2 Componentes utilizados

A simulação foi organizada com base no ESP32 como unidade central de controle, associado a um conjunto de sensores e componentes auxiliares capazes de representar um *smartwatch* de saúde de forma coerente com a proposta da atividade. Os principais elementos utilizados foram os seguintes.

a) ESP32

O ESP32 foi utilizado como microcontrolador principal do sistema. Sua função é coordenar a leitura dos sensores, processar os dados localmente, gerenciar o estado de conectividade simulada, controlar a lógica de armazenamento local e executar a sincronização das informações quando a conexão é restabelecida.



b) Sensor DHT22

O DHT22 foi utilizado como sensor obrigatório da atividade. Ele fornece leituras de temperatura e umidade. No contexto do projeto, a temperatura representa uma variável de saúde relevante, enquanto a umidade foi mantida como dado complementar de contexto ambiental. A inclusão desse sensor atende diretamente ao requisito mínimo do enunciado e amplia a qualidade do monitoramento simulado.

c) Sensor MPU60504

O MPU6050 foi utilizado para representar o monitoramento de movimento. Como esse sensor reúne acelerômetro e giroscópio, ele permite simular a atividade motora do usuário, incluindo cenários de movimentação, baixa atividade ou possíveis alterações bruscas. Sua presença aproxima o protótipo do comportamento esperado de dispositivos vestíveis modernos, que normalmente acompanham atividade física e estado de repouso.

d) Potenciômetro

O potenciômetro foi adotado como recurso de simulação para frequência cardíaca. Embora não constitua um sensor biomédico real, ele oferece uma solução didática e funcional no Wokwi, permitindo variar manualmente o valor de entrada e representar mudanças de batimentos cardíacos. Esse componente foi particularmente útil para construção de cenário de alertas, como taquicardia simulada.

e) Botão de emergência

O botão de emergência foi incluído para representar um acionamento manual de evento crítico. Sua função no projeto é simular situação em que o usuário solicita ajuda, registra uma condição anormal ou dispara um alerta imediato por decisão própria. Esse elemento contribui para aproximar o protótipo de cenários reais de dispositivos voltados à segurança e ao monitoramento remoto.

f) *LED* de alerta

O *LED* foi utilizado como mecanismo de sinalização visual local. Sempre que uma condição de risco é detectada pelo sistema, o *LED* é acionado, representando uma resposta rápida no nível do dispositivo. Assim, o



protótipo passa a incorporar não apenas aquisição de dados, mas também uma camada local de alerta.

g) Módulo micros

O módulo micros foi adotado como mecanismo principal de armazenamento local. Sua função é registrar temporariamente as leituras enquanto o sistema se encontra sem conectividade simulada. Essa escolha permite representar de forma mais concreta o conceito de persistência local, mantendo os dados disponíveis até o momento de sincronização.

2.3 Estrutura lógica de funcionamento

O sistema foi projetado para operar de forma cíclica. Em intervalos previamente definidos, o ESP32 realiza uma nova leitura dos sensores e constrói um registro contendo os dados atuais do estado do usuário e do dispositivo. Cada registro é estruturado com os seguintes campos:

- Tempo da leitura em milissegundos;
- Temperatura;
- Umidade;
- Frequência cardíaca simulada;
- Presença ou ausência de movimento;
- Estado do botão de emergência.

Depois de capturar essas informações, o sistema executa uma etapa local de processamento, na qual verifica se existem condições que exigem sinalização de alerta. Caso algum limite seja ultrapassado, o *LED* do sistema é acionado e uma mensagem correspondente é exibida no Monitor Serial. Essa lógica demonstra que o dispositivo não depende exclusivamente de processamento externo para reagir a eventos relevantes.

A aplicação foi organizada com funções específicas para inicialização, leitura de sensores, criação de registro, avaliação de alertas, armazenamento local, sincronização e impressão das informações no Monitor Serial. Essa separação em



blocos melhora a legibilidade do código, facilita manutenção e permite evolução posterior do sistema.

2.4 Regras de alerta implementadas

A etapa de processamento local inclui uma lógica simples de avaliação de risco. Foram definidas regras básicas para representar situações críticas ou anormais em um smartwatch de saúde. Entre elas, destacam-se:

Temperatura acima do limite definido como seguro;

Frequência cardíaca simulada acima de faixa de referência configurada;

Botão de emergência pressionado.

Quando qualquer uma dessas condições ocorre, o sistema interpreta o evento como alerta ativo. Nesse caso, o *LED* é acionado e uma mensagem correspondente é exibida. Ainda que a lógica empregada seja inicial, ela demonstra claramente o papel do processamento local em aplicações de borda: detectar eventos relevantes sem depender, naquele instante, de envio ou análise remota.

2.5 Armazenamento local com micros

Considerando o requisito da atividade relacionada ao armazenamento local, foi implementado o uso do módulo microSD como mecanismo principal para retenção temporária dos dados. Durante o estado offline, as leituras não são descartadas. Em vez disso, são registradas em um arquivo no cartão, no formato CSV.

A escolha de formato CSV foi adequada por sua simplicidade de implementação, facilidade de leitura e compatibilidade futura com ferramentas de análise, exportação e tratamento de dados. O arquivo contém cabeçalho e registros estruturados com todos os campos relevantes da leitura. Assim, o sistema mantém uma base organizada de informações pendentes, pronta para sincronização quando a conectividade for restabelecida.

A utilização do microSD representa uma solução coerente com a ideia de armazenamento persistente no nível do dispositivo, aproximando o protótipo de uma implementação real. Ao contrário do uso exclusivo de memória volátil, o



armazenamento em arquivo reforça a confiabilidade da retenção local durante a indisponibilidade da rede.

2.6 Buffer em RAM como fallback e demonstração

Além do microSD, foi mantido um *buffer* circular em memória *RAM* como estratégia complementar. Essa *buffer* funciona como mecanismo de *fallback* e também como recurso demonstrativo dos conceitos de *Edge Computing*. Sua função é garantir que, caso o microSD não esteja disponível ou apresente falha, o sistema continue armazenando temporariamente as leituras em memória.

A estrutura escolhida para esse *buffer* foi tipo *FIFO*, ou seja, “*First In, First Out*”. Isso significa que, quando o limite configurado é atingindo, os registros mais antigos são removidos para dar lugar aos mais novos. Essa política é apropriada para sistemas de monitoramento contínuo, nos quais as informações mais recentes tendem a ser mais relevantes para a análise imediata.

A presença do *buffer* em *RAM* torna a arquitetura mais robusta, pois evita que a coleta seja interrompida por falha em um único meio de armazenamento. Além disso, do ponto de vista didático, esse componente reforça a demonstração de resiliência offline exigida pela atividade.

2.7 Estratégia de resiliência offline

A principal característica da primeira etapa do Cardiola é a implementação de resiliência offline. Em sistemas críticos de monitoramento, a indisponibilidade temporária de rede não pode impedir a continuidade da coleta de dados. Por isso, foi simulada uma variável booleana de conectividade, representando os estados online e offline.

Quando o sistema se encontra desconectado, as leituras seguem sendo realizadas normalmente. A diferença é que, nesse estado, os registros deixam de ser enviados e passam a ser armazenados localmente. A prioridade é dada ao microSD. Se o módulo estiver operacional, o dado é gravado em arquivo. Caso contrário, o registro é encaminhado para o *buffer* em *RAM*.



Essa estratégia garante que o sistema preserve informações mesmo em ambiente sem comunicação, mantendo a lógica de continuidade operacional própria da computação em borda. Em aplicações reais de saúde, esse comportamento é particularmente importante, pois evita perda de dados em contextos nos quais a conectividade pode variar.

2.8 Sincronização dos dados

Quando a conectividade simulada é restabelecida, o sistema inicia o processo de sincronização. Nessa etapa, os dados armazenados localmente são enviados de maneira simulada por meio do *Serial.println()*, conforme permitido pelo enunciado da atividade.

Primeiramente, a aplicação transmite a leitura atual. Em seguida, processa os registros armazenados no microSD e, depois, os registros mantidos no *buffer* em RAM. Após o envio dos dados persistidos no cartão, o arquivo local é limpo e recriado com cabeçalho, os registros presentes no *buffer* em memória são removidos à medida que são sincronizados.

Essa lógica reproduz o comportamento esperado de uma aplicação *IoT* com computação em borda: capturar dados, protegê-los localmente em caso de falha de rede e sincronizá-los quando a comunicação volta a estar disponível. Ainda que a nuvem real não tenha sido implementada nesta etapa, a estrutura já se encontra alinhada com as próximas fases do projeto, especialmente com o envio via *MQTT* e a visualização em dashboards.

2.9 Organização do código e boas práticas adotadas

O código da aplicação foi estruturado com foco em clareza, escalabilidade e reaproveitamento futuro. As variáveis foram nomeadas em inglês, facilitando tanto a padronização técnica quanto a compreensão do projeto no contexto acadêmico.

Também foi adotado o uso de tipos inteiros de tamanho definido, como *uint8_t*, *uint16_t* e *uint32_t*, a fim de tornar o uso de memória mais explícito e coerente com boas práticas de programação embarcada. A divisão do programa em funções



específicas permite evolução gradual do protótipo, como substituição da sincronização por *MQTT*, integração com *APIs*, inclusão de novos sensores e aplicações de rotinas mais avançadas de análise.

A arquitetura atual já permite expansão para as próximas etapas do CardiolA sem necessidade de reestruturação completa do projeto, o que demonstra que a primeira fase foi desenvolvida não apenas para comprimir o requisito imediato, mas também para servir de base para o restante da solução.

2.10 Justificativa para o não uso do SPIFFS e adoção de meio alternativo

Embora o *SPIFFS* tenha sido considerado inicialmente como opção de armazenamento local para o projeto, sua utilização direta não foi adotada como solução principal nesta etapa. Em ambiente real com ESP32, o *SPIFFS* é um sistema de arquivos otimizado para memória flash *SPI NOR*, apropriado para pequenas partições e baixas taxas de transferência, sendo uma alternativa válida para retenção local de dados em dispositivos embarcados.

Entretanto, no contexto do simulador Wokwi, optou-se por não centralizar a solução no *SPIFFS*. Isso ocorreu porque a proposta do projeto exigia uma demonstração mais concreta do armazenamento local, da retenção temporária de leituras e da posterior sincronização. Para atender a esse objetivo com maior clareza, foi adotado o módulo microSD como meio principal de armazenamento local, uma vez que o Wokwi documenta suporte direto ao cartão microSD com sistema de arquivos *FAT16*, capacidade simulada de até 8 MB e manipulação de arquivos no contexto do projeto.

Essa escolha apresenta uma diferença importante entre o ambiente de simulação e a aplicação em hardware real. No mundo real, o *SPIFFS* opera diretamente sobre a flash do ESP32, integrado à lógica interna do dispositivo e orientado ao uso embarcado. Já o microSD representa um meio externo e removível, baseado em interface *SPI* e sistema de arquivos *FAT*, com comportamento mais próximo de armazenamento persistente convencional. Assim, enquanto o *SPIFFS* seria mais aderente à arquitetura interna do ESP32 físico, o microSD mostrou-se mais



adequado para fins de simulação, visualização prática dos registros e demonstração do fluxo de gravação e leitura no Wokwi.

Além disso, foi mantido no projeto um *buffer* em memória *RAM* como mecanismo complementar de *fallback* e demonstração. Dessa forma, caso o armazenamento no microSD não esteja disponível, o sistema continua preservando temporariamente as leituras em memória, garantindo a continuidade da coleta e reforçando o conceito de resiliência offline. Essa decisão não substitui a persistência real do armazenamento em flash ou cartão, mas demonstra de forma clara o princípio da computação em borda, no qual o dispositivo continua operando e protegendo os dados mesmo sem conectividade imediata. Essa conclusão decorre da arquitetura implementada no protótipo e da comparação entre os recursos disponíveis no simulador e os mecanismos típicos do ESP32 físico.

Portanto, o não uso do *SPIFFS* como solução principal não representa uma limitação conceitual do projeto, mas sim uma decisão técnica e metodológica. Em uma implementação física com ESP32 real, o *SPIFFS* poderia ser empregado de forma mais aderente ao hardware embarcado. Já no ambiente simulado, o uso do microSD permitiu uma demonstração mais robusta, observável e compatível com os objetivos da atividade, enquanto o *buffer* em *RAM* complementou a lógica de tolerância a falhas e continuidade operacional.



3. CONCLUSÃO

A primeira etapa do projeto CardiolA permitiu construir uma base funcional e tecnicamente coerente para um sistema vestível de monitoramento aplicado à saúde digital. Por meio do uso do ESP32, de sensores representativos e de uma lógica de *Edge Computing*, foi possível implementar um protótipo capaz de coletar sinais relevantes, processá-los localmente, armazená-los temporariamente e sincroniza-los posteriormente.

O uso do microSD como armazenamento principal agregou maior realismo à proposta, pois permitiu representar a persistência local dos dados de forma mais próximas de um sistema real. Ao mesmo tempo, a manutenção de um buffer em *RAM* como *fallback* fortaleceu a robustez de aplicações, demonstrando uma estratégia complementar de tolerância a falhas. Essa arquitetura híbrida reforça a preocupação com continuidade operacional, aspecto essencial em sistemas de monitoramento crítico.

Além de atender aos requisitos da atividade, a solução desenvolvida estabelece uma base consistente para evolução futura. A lógica já se encontra preparada para integração com *MQTT*, dashboards em *Node-RED*, automação de alertas e possíveis etapas de análise inteligente. Conclui-se, portanto, que esta fase do CardiolA cumpriu seu papel de demonstrar, de forma prática e fundamentada, a relevância da computação em borda em aplicações *IoT* voltadas à saúde.



FONTES

Wokwi-dht22 Reference.

Disponível em: https://docs.wokwi.com/parts/wokwi-dht22?utm_source=chatgpt.com. Acesso em 23/04/2026.

Wokwi-mpu6050 6-Axis Accel & Gyro Sensor.

Disponível em: https://docs.wokwi.com/parts/wokwi-mpu6050?utm_source=chatgpt.com. Acesso em 23/04/2026.

Wokwi-microsd-card Reference.

Disponível em: https://docs.wokwi.com/parts/wokwi-microsd-card?utm_source=chatgpt.com. Acesso em 23/04/2026.

ESP-IDF Programming Guide – SPIFFS Filesystem.

Disponível em: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/storage/spiffs.html?utm_source=chatgpt.com. Acesso em 23/04/2026.

ESP-IDF Programming Guide – Storage API.

Disponível em: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/storage/index.html?utm_source=chatgpt.com. Acesso em 23/04/2026.

ESP-IDF Programming Guide – FreeRTOS (Supplemental Features).

Disponível em: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_additions.html. Acesso 23/04/2026.

Smartwatches e medidas de saúde: a tecnologia a favor da medicina.

Disponível em: <https://profissaobiotec.com.br/smartwatches-medidas-saude-tecnologia-favor-medicina/>. Acesso em 23/04/2026.

Para que servem os sensores de um Smartwatch.

Disponível em: https://canaltech.com.br/smartwatch/para-que-servem-os-sensores-de-um-smartwach/#google_vignette. Acesso em 23/04/2026.

Faculdade de Informática e Administração Paulista (FIAP). Código Organizado, Projeto Seguro: POO Na Prática Com Arduino e ESP32. São Paulo: 2025.

Faculdade de Informática e Administração Paulista (FIAP). IoT Cognitivo: Que Falam e Pensam. São Paulo: 2026.

Chris CDFER. Ring-Buffer-Demo-ESP32-Arduino. 2024. Disponível em:

<https://github.com/CDFER/Ring-Buffer-Demo-ESP32-Arduino?tab=readme-ov-file>. Acesso em: 23/04/2026

